



The Knife

Manuale tecnico

Progetto	Laboratorio Interdisciplinare A
Autori	Matteo Franguelli, Elia Toschi, Celestino Resteghini, Michele Viselli
Sede	Università degli Studi dell'Insubria – Varese
Data	31 agosto 2026
Versione documento	2.0

Indice

1. Installazione

1.1 Requisiti di sistema

1.2 Setup ambiente

1.3 Installazione programma

2. Esecuzione ed uso

2.1 Setup e lancio del programma

2.2 Uso delle funzionalità

2.2.1 Struttura dell'applicazione

2.2.2 Scelte architettureali

2.2.3 Struttura statica: class diagram

2.2.4 Struttura dinamica: sequence e state diagram

2.2.5 Protocollo client/server

2.2.6 Progettazione della base di dati

2.2.7 Strutture dati utilizzate

2.2.8 Scelte algoritmiche

2.2.9 Design pattern adottati

2.2.10 Documentazione Javadoc

2.3 Data set di test

3. Limiti della soluzione sviluppata

4. Sitografia / Bibliografia

1. Installazione

TheKnife è una piattaforma per cercare e consultare ristoranti, sulla scia di TheFork: il catalogo si filtra per città, tipologia di cucina, fascia di prezzo, valutazione media, consegna a domicilio e prenotazione in linea. La prima versione era monolitica e teneva i dati in file CSV locali. Questa è stata riprogettata come **applicazione client/server a tre livelli**: un client JavaFX, un server multi-thread e una base di dati PostgreSQL. Il capitolo descrive la catena di build, le dipendenze e l'installazione dei due eseguibili.

1.1 Requisiti di sistema

Il progetto è un *multi-module Maven project* compilato con `--release 21` e distribuito come coppia di *fat jar* prodotti dal `maven-shade-plugin`. Lo sviluppo e il collaudo sono avvenuti su Windows 10/11, ma nel codice non c'è nulla di legato alla piattaforma: servono soltanto la JVM e i binari nativi di JavaFX per l'architettura in uso.

Componente	Versione e note
JDK / JRE	Da 21 a 25; la 21 è la <i>release target</i> dichiarata nel POM padre
JavaFX	21 (<code>javafx-controls</code> , <code>javafx-fxml</code> , <code>javafx-web</code>)
Apache Maven	3.9 o superiore; nel repository è incluso il wrapper <code>mvnw</code>
PostgreSQL	12 o superiore, collaudato sulla 18; driver JDBC <code>org.postgresql:postgresql:42.7.3</code>
Flyway	<code>flyway-core</code> e <code>flyway-database-postgresql</code> 10.15.0, per il versionamento dello schema
OpenCSV	5.9, usata dal solo modulo server per l'importazione iniziale
Memoria	Almeno 256 MB di heap per il client; il server è vincolato dal database

Limite superiore sulla versione della JVM. Il client usa `javafx.web` per incorporare il sito dei ristoranti in una `WebView`, e quel modulo richiede `jdk.jsobject`, rimosso dal JDK a partire dalla release 26. Con un JDK 26 la risoluzione del *module graph* fallisce all'avvio con `FindException: Module jdk.jsobject not found, required by javafx.web`, prima di eseguire una sola istruzione dell'applicazione.

1.2 Setup ambiente

La radice del repository è un aggregatore con `packaging` di tipo `pom` che dichiara i tre moduli applicativi. Il coordinato del progetto è `it.uninsubria:TheKnife:4.0` e ogni modulo eredita da esso versione, encoding e livello del compilatore.

Modulo	Responsabilità	Dipendenze dichiarate
commonTK	Contratto condiviso: i DTO serializzabili scambiati sul socket e le validazioni comuni ai due lati; qui sono centralizzate le dipendenze JavaFX	JavaFX <code>controls</code> , <code>fxml</code> e <code>web</code>
serverTK	Accettazione delle connessioni, protocollo, accesso al database tramite DAO, creazione dello schema e importazione dei dati	<code>commonTK</code> , driver JDBC PostgreSQL, Flyway, OpenCSV
clientTK	Interfaccia grafica JavaFX, controller delle viste FXML, sessione e richieste al server	<code>commonTK</code>

JavaFX è dichiarato una volta sola, in `commonTK`, e arriva agli altri due moduli per transitività: serve al client per l'intera interfaccia e al server per la finestra di avvio. Le direttive `requires` restano invece in ciascun descrittore JPMS, perché riguardano i moduli effettivamente usati dal codice di quel modulo.

`clientTK` e `serverTK` dipendono entrambi da `commonTK`, che deve quindi trovarsi nel *repository locale* prima di compilare un modulo per conto suo. La build dell'aggregatore lo installa da sola, costruendo i moduli nell'ordine dettato dalle dipendenze.

Le credenziali del database non stanno nel codice né in un file di configurazione: host, porta, utente e password si indicano nella finestra di avvio del server, che li passa a `DatabaseConfig.configura(...)`. In mancanza di indicazioni valgono i valori di default, `localhost:5432` con utente `postgres`.

```
public static void configura(String host, String porta, String utente, String password) {
    props.setProperty("db.url.default", "jdbc:postgresql://" + host + ":" + porta + "/postgres");
    props.setProperty("db.user", utente);
    props.setProperty("db.password", password);
}
```

Il bootstrap della persistenza è automatico e idempotente. `inizializzaDatabaseCompleto()` si collega al database di sistema `postgres`, controlla su `pg_database` se `theknife_db` esiste e lo crea in caso contrario; poi lascia a Flyway le migrazioni e avvia l'importazione del data set. Nessuna operazione manuale di DDL: le tabelle nascono dallo script versionato `V1__creazione_tabelle.sql`, che sta in `serverTK/src/main/resources/db/migration` secondo la convenzione di Flyway.

```
Flyway flyway = Flyway.configure()
    .dataSource(getTargetUrl(), getUser(), getPassword())
    .load();
flyway.migrate();
```

1.3 Installazione programma

La fase `package` produce due archivi autoconsistenti, uno per il client e uno per il server. Il `maven-shade-plugin` vi include le dipendenze di terze parti e scrive nel manifest la classe di avvio: `theknife.TheKnife` per il client, `it.uninsubria.AppServer` per il server. Insieme agli archivi vengono distribuiti gli script `avvioWindows.bat` e `avvioMacOS.command`, che lanciano il server, attendono la porta e aprono il client.

Il punto di ingresso del client. Non è `MainApp` ma `theknife.TheKnife`, che si limita a chiamare `Application.launch(MainApp.class, args)`. La JVM rifiuta infatti di avviare direttamente una sottoclasse di `Application` quando i moduli JavaFX stanno sul classpath, come accade in un archivio prodotto per *shading*.

2. Esecuzione ed uso

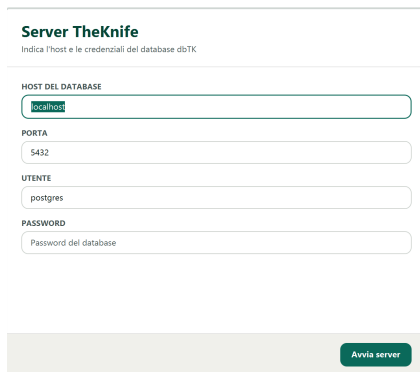
2.1 Setup e lancio del programma

Il sistema si compone di due processi distinti. Il processo server apre un `ServerSocket` sulla porta **8999** e per ogni connessione accettata avvia un thread dedicato; il processo client apre una singola connessione TCP verso di esso e vi instrada tutte le richieste. Il client non possiede alcuna credenziale del database e non apre mai una connessione JDBC: l'accesso alla base di dati è confinato nel modulo server.

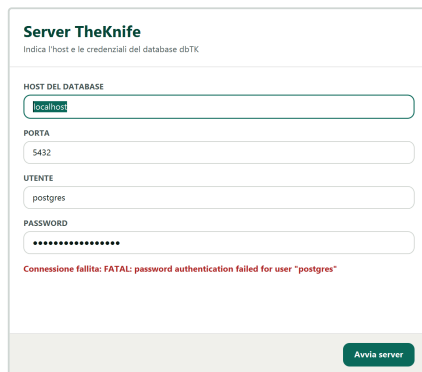
```
public static void exec() throws IOException {
    ServerSocket s = new ServerSocket(8999);
    try {
        while (true) {
            Socket socket = s.accept();
            new SlaveThread(socket).start();
        }
    } catch (ClassNotFoundException e) {
        throw new RuntimeException(e);
    } finally {
        s.close();
    }
}
```

L'ordine di avvio è vincolante: prima il server, perché il client chiede l'elenco delle città già mentre inizializza la schermata di benvenuto.

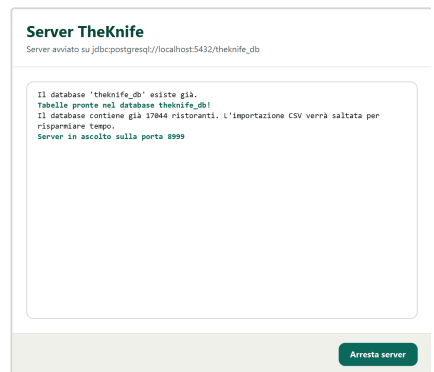
Anche la finestra del server è un'applicazione JavaFX in MVC: `ServerApp` come classe `Application`, `server.fxml` come vista, `ServerController` come controller. Raccoglie host, porta, utente e password, prova la connessione e, se riesce, avvia il ciclo di `accept` su un thread separato. `System.out` viene dirottato su un `TextFlow` che fa da registro, con le righe colorate secondo il tipo di messaggio.



Parametri di connessione al cluster PostgreSQL.



Eccezione JDBC riportata all'operatore.



Registro del server in ascolto sulla porta 8999.

Sul lato client la connessione sta tutta in `GestioneRichieste`, che apre il socket verso `localhost:8999` e tiene aperti per l'intera sessione un `ObjectOutputStream` e un `ObjectInputStream`. La coppia si crea una volta sola, e l'ordine conta: l'`ObjectOutputStream` scrive l'intestazione di serializzazione alla costruzione, l'`ObjectInputStream` la legge. Invertirlo fra i due estremi manda in stallo l'handshake. Alla chiusura della finestra principale `MainApp` invia `END` e chiude il socket.

Sintomo	Causa tecnica e rimedio
FindException: Module jdk.jsobject not found	JVM 26 o superiore: il modulo richiesto da <code>javafx.web</code> non esiste più. Installare un JDK fra la 21 e la 25.
ConnectException: Connection refused all'avvio del client	Il server non è in ascolto sulla porta 8999, oppure la porta è filtrata dal firewall.
La finestra del server segnala un errore di autenticazione	Credenziali PostgreSQL errate o servizio non attivo; l'eccezione JDBC compare per intero nel riquadro dei messaggi.

Il primo avvio impiega alcuni minuti	È in corso l'importazione del catalogo: circa 17.700 righe inserite con <code>PreparedStatement</code> . Gli avvii successivi rilevano il database popolato e saltano la procedura.
BUILD SUCCESS senza ricompilazione dopo una modifica	La build è stata lanciata su un modulo diverso da quello modificato: nel log deve comparire <code>Compiling N source files</code> per il modulo atteso.

2.2 Uso delle funzionalità

2.2.1 Struttura dell'applicazione

I tre moduli Maven sono anche tre moduli JPMS con lo stesso nome, e ciascuno dichiara il proprio descrittore. Il client apre alla riflessione i soli package che contengono i controller, come chiede il caricatore FXML.

```
module clientTK {
    requires commonTK;
    requires javafx.controls;
    requires javafx.fxml;
    requires java.desktop;
    requires javafx.web;

    exports theknife;
    exports theknife.ui.javaafx;
    opens theknife.ui.javaafx to javafx.fxml;
    exports theknife.utilities;
    opens theknife.utilities to javafx.fxml;
}
```

Il descrittore di `serverTK` aggiunge `opens db.migration;`. Flyway e `ImportaDati` leggono le migrazioni e i CSV attraverso il *class loader*, e le risorse di un modulo nominato restano incapsulate finché il package non viene aperto. Fra i `requires` compaiono anche i moduli che Flyway carica per conto proprio via `ServiceLoader`, come `com.google.gson` e i moduli Jackson: sul module path un modulo esplicito viene risolto solo se qualcuno lo richiede.

Package	Contenuto
it.uninsubria.dto (commonTK)	Dieci DTO serializzabili: <code>UtenteDTO</code> , <code>RistoratoreDTO</code> , <code>RistoranteDTO</code> , <code>RecensioneDTO</code> , <code>RispostaDTO</code> , <code>LuogoDTO</code> , <code>CittaDTO</code> , <code>CoordinateDTO</code> , <code>FiltroRistoranteDTO</code> , <code>AuthDTO</code>
it.uninsubria (serverTK)	<code>AppServer</code> , <code>SlaveThread</code> , <code>ServerApp</code> , <code>ServerController</code> , <code>DatabaseConfig</code> , <code>ImportaDati</code> , <code>Utility</code>
it.uninsubria.dao (serverTK)	<code>UtenteDAO</code> , <code>RistoranteDAO</code> , <code>RecensioneDAO</code> , <code>PreferitiDAO</code> , <code>LuogoDAO</code>
theknife.ui.javaafx (clientTK)	Un controller per ciascuna vista FXML, più <code>Session</code> e l'interfaccia <code>ControllerAutenticazione</code>
theknife.utilities (clientTK)	<code>Finestre</code> , <code>Temi</code> , <code>Etichette</code> , <code>Utility</code>
theknife.model (clientTK)	<code>GestioneRichieste</code> , unico punto di accesso alla rete

Le risorse del client — FXML, fogli di stile e immagini — stanno sotto `it/uninsubria/theknifeui/ui/javaafx/`. L'albero delle risorse è quindi disallineato rispetto a quello dei sorgenti, e le viste si risolvono con percorsi assoluti tramite `getClass().getResource(...)`.

2.2.2 Scelte architetturali

L'architettura è stratificata su tre livelli, con dipendenze in una sola direzione:

- **presentazione** — i controller JavaFX, che non contengono istruzioni SQL né codice di rete al di fuori delle chiamate a `GestioneRichieste`;
- **applicativo** — `SlaveThread`, che interpreta il protocollo, deserializza il payload, invoca il DAO competente e serializza la risposta;

- **persistenza** – i DAO, che incapsulano ogni interazione JDBC e restituiscono esclusivamente DTO, mai `ResultSet` aperti.

La concorrenza segue il modello *thread per connessione*: il thread principale esegue solo l'`accept`, ogni client ha il proprio `SlaveThread`. Più utenti lavorano quindi in parallelo, e lo stato locale del thread tiene separate le sessioni. La coerenza dei dati resta al DBMS: ogni operazione apre e chiude la propria connessione JDBC in un blocco `try-with-resources` e si esaurisce in una transazione implicita.

Il livello di presentazione applica il pattern MVC nella declinazione di JavaFX: la vista è dichiarata in FXML, il controller viene iniettato dal `FXMLLoader` nei campi annotati con `@FXML`, il modello sono i DTO che arrivano dal server. Anche l'interfaccia del server segue la stessa impostazione: pur avendo una sola schermata, ha un FXML e un controller dedicati invece di costruire i nodi a mano nella classe `Application`.

2.2.3 Struttura statica: class diagram

Il diagramma delle classi mostra la struttura statica a un livello più alto del codice. Compaiono le classi significative con i soli attributi e le sole operazioni che contano per le collaborazioni; accessori, costruttori e metodi privati di supporto restano fuori. Le tre partizioni verticali sono i tre moduli Maven e rendono evidente la direzione delle dipendenze: entrambi i lati conoscono `commonTK`, mentre fra `clientTK` e `serverTK` non esiste alcun arco diretto. Si parlano solo attraverso il protocollo su socket.

I tre riquadri qui sotto isolano altrettante classi significative; il diagramma completo è alla pagina seguente.

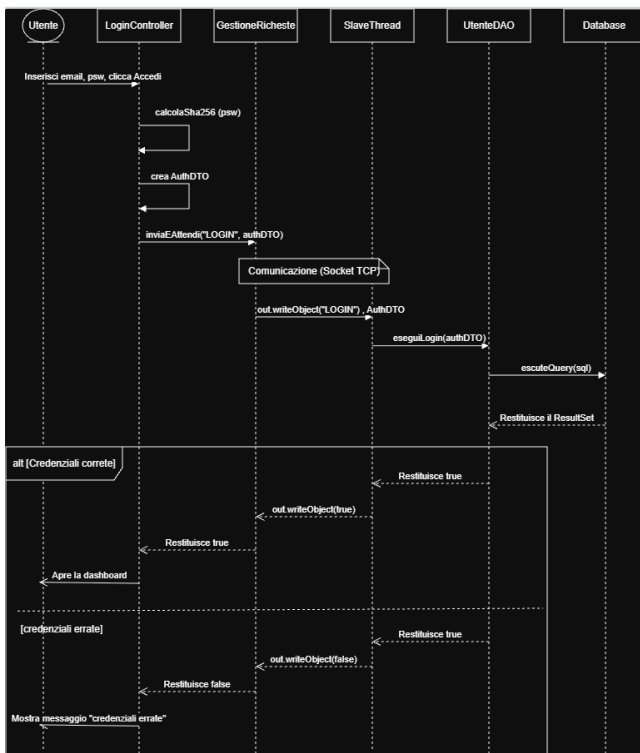


`GestioneRichieste` e `SlaveThread` sono simmetrici: detengono le due estremità della medesima coppia di stream. I DTO invece si compongono: `RistoranteDTO` contiene un `LuogoDTO`, che a sua volta contiene `CittaDTO` e `CoordinateDTO`. Una singola risposta trasporta così tutto il grafo che serve a comporre una scheda, senza richieste successive. `RistoratoreDTO`, infine, estende `UtenteDTO` e riporta sul contratto condiviso il vincolo di ruolo dello schema concettuale.

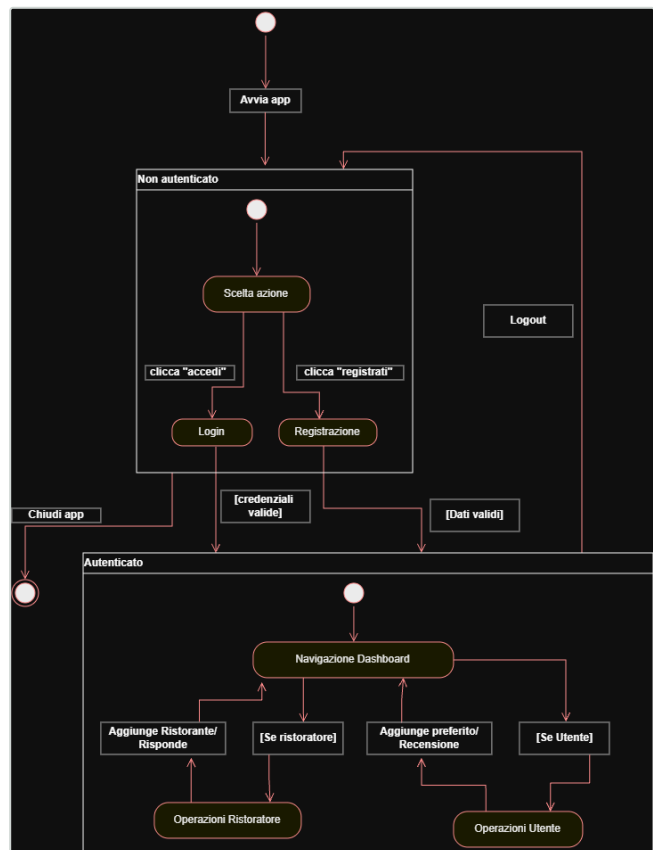
2.2.4 Struttura dinamica: sequence e state diagram

Due diagrammi raccontano il comportamento del sistema. Il sequence diagram segue un caso d'uso rappresentativo, l'accesso di un utente registrato, e attraversa tutti i livelli. Il controller della vista calcola il digest della password e costruisce l'AuthDTO; `GestioneRichieste` lo serializza sul socket; `SlaveThread` lo deserializza e delega a `UtenteDAO`, che interroga il database. Il frammento combinato *alt* distingue i due esiti, entrambi un booleano che torna indietro lungo la stessa catena. L'accesso è un esempio significativo, non un caso particolare: ogni altra operazione segue lo stesso percorso, con la stessa architettura, cambiando soltanto il comando e i DTO scambiati.

Lo state diagram descrive il ciclo di vita della sessione, che il singleton `Session` tiene in memoria. Dallo stato composito *Non autenticato* si passa ad *Autenticato* solo con credenziali valide o dopo una registrazione; il logout riporta la macchina allo stato iniziale, azzerando ruolo, permessi, città e identificativo. Dentro lo stato autenticato le transizioni dipendono dal ruolo, secondo le guardie riportate sugli archi.

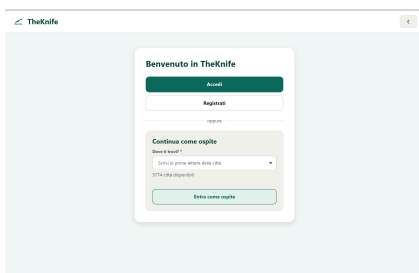


Sequence diagram: autenticazione di un utente registrato.

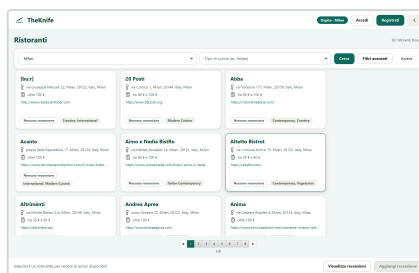


State diagram: ciclo di vita della sessione e transizioni per ruolo.

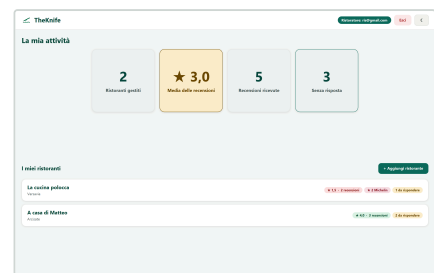
Sul piano dell'interfaccia, a ogni transizione corrisponde la sostituzione del contenuto della finestra principale: la stessa *Scene* viene riutilizzata caricando la vista adatta al ruolo restituito dal server, senza creare nuovi *Stage*.



Stato Non autenticato: vista di benvenuto.



Stato autenticato, ruolo cliente o ospite.



Stato autenticato, ruolo ristorante.

2.2.5 Protocollo client/server

Sopra il trasporto TCP corre un protocollo applicativo con comandi testuali e payload binari, affidati alla serializzazione Java. Ogni richiesta è una stringa che identifica il comando, seguita se serve da un oggetto

argomento; quelle che producono un risultato restano in attesa della risposta. La stringa **END** chiude la sessione e termina il ciclo del thread servente.

```
public synchronized Object inviaEAttendi(String comando, Object dato) {
    try {
        out.writeObject(comando);
        out.writeObject(dato);
        out.flush();
        return in.readObject();
    } catch (IOException | ClassNotFoundException e) {
        e.printStackTrace();
        return null;
    }
}
```

I metodi di invio sono **synchronized** perché il client condivide un solo socket fra il thread applicativo JavaFX e i thread di lavoro che eseguono le richieste bloccanti. Senza mutua esclusione due richieste concorrenti intreccerebbero le scritture sullo stream, disallineando per sempre richieste e risposte. Sul lato server il ciclo di ascolto instrada ogni comando al proprio gestore.

```
while (true) {
    comando = (String) in.readObject();
    if (comando.equals("END")) break;

    switch (comando) {
        case "LOG"    -> gestisciLogin();
        case "REG"    -> gestisciRegistrazione();
        case "FILTRO" -> gestisciFiltro();
        case "REC"    -> gestisciRecensioni();
        /* ... */
        default -> System.err.println("Comando sconosciuto: " + comando);
    }
}
```

I venti comandi si dividono in due famiglie. Quelli che restituiscono dati viaggiano con **inviaEAttendi** ; quelli di sola scrittura con **inviaSolo** , senza risposta, perché il client ricarica comunque i dati interessati dopo l'operazione.

Comandi generali

Comando	Il client invia	Il server risponde
END	–	Nessuna risposta: chiude la connessione fra client e server
CITTA	–	List<String> con i nomi di tutte le città
CUC	–	List<String> con i nomi di tutte le cucine
ID	Email dell'utente	Identificativo dell'utente

Ospite e utente non ancora registrato

Comando	Il client invia	Il server risponde
LOG	AuthDTO	HashMap<String, Boolean> con l'esito dell'accesso e la tipologia di utente
REG	UtenteDTO	Boolean con l'esito della registrazione
FILTRO	FiltroRistoranteDTO	Lista dei ristoranti che soddisfano i criteri
REC	Identificativo del ristorante	Lista delle recensioni di quel ristorante

Cliente

Comando	Il client invia	Il server risponde
AGG-REC	RecensioneDTO	Nessuna risposta
VIS-REC	Identificativo dell'utente	Lista delle recensioni scritte dall'utente

MOD-REC	RecensioneDTO	Nessuna risposta
ELIM-REC	Identificativo della recensione	Nessuna risposta
VIS-PREF	Identificativo dell'utente	Lista dei ristoranti preferiti
AGG-PREF	HashMap con identificativo dell'utente e del ristorante	Nessuna risposta
ELIM-PREF	HashMap con identificativo dell'utente e del ristorante	Nessuna risposta
MAPS	Identificativo del ristorante	CoordinatedTO per l'apertura in Google Maps
DOM	Email dell'utente	String con la città di domicilio

Ristoratore

Comando	Il client invia	Il server risponde
RIST	Identificativo del ristorante	Lista dei ristoranti che gestisce
AGG-RIST	RistoranteDTO	Nessuna risposta
REC-NO-RISP	Identificativo del ristorante	Lista delle recensioni ancora senza risposta
RISP-REC	RispostaDTO	Nessuna risposta

I criteri della maschera confluiscono in **FiltroRistoranteDTO** (comando **FILTRO**).

Inserimento di una recensione: comando **AGG-REC**.

Risposta del ristorante: comando **RISP-REC**.

2.2.6 Progettazione della base di dati

Lo schema concettuale esiste in due versioni. La prima riflette l'analisi dei requisiti e contiene costrutti che il modello relazionale non traduce direttamente; la seconda è il risultato della ristrutturazione, che li elimina senza perdere semantica.

La traduzione logica produce undici relazioni. Entità deboli e associazioni reificate diventano tabelle con chiave primaria composta. L'integrità referenziale è dichiarata con chiavi esterne in propagazione (**ON UPDATE CASCADE** e **ON DELETE CASCADE**): cancellando un ristorante spariscono anche le sue recensioni, le risposte e i riferimenti nei preferiti, senza codice applicativo di pulizia.

```
CREATE TABLE RISPOSTA (
    id_recensione INT NOT NULL UNIQUE REFERENCES RECENSIONE(id)
                                ON UPDATE CASCADE ON DELETE CASCADE,
    id_utente      INT NOT NULL REFERENCES UTENTE(id_utente)
                                ON UPDATE CASCADE ON DELETE CASCADE,
    testo_risposta TEXT NOT NULL,
    PRIMARY KEY (id_recensione)
);
```

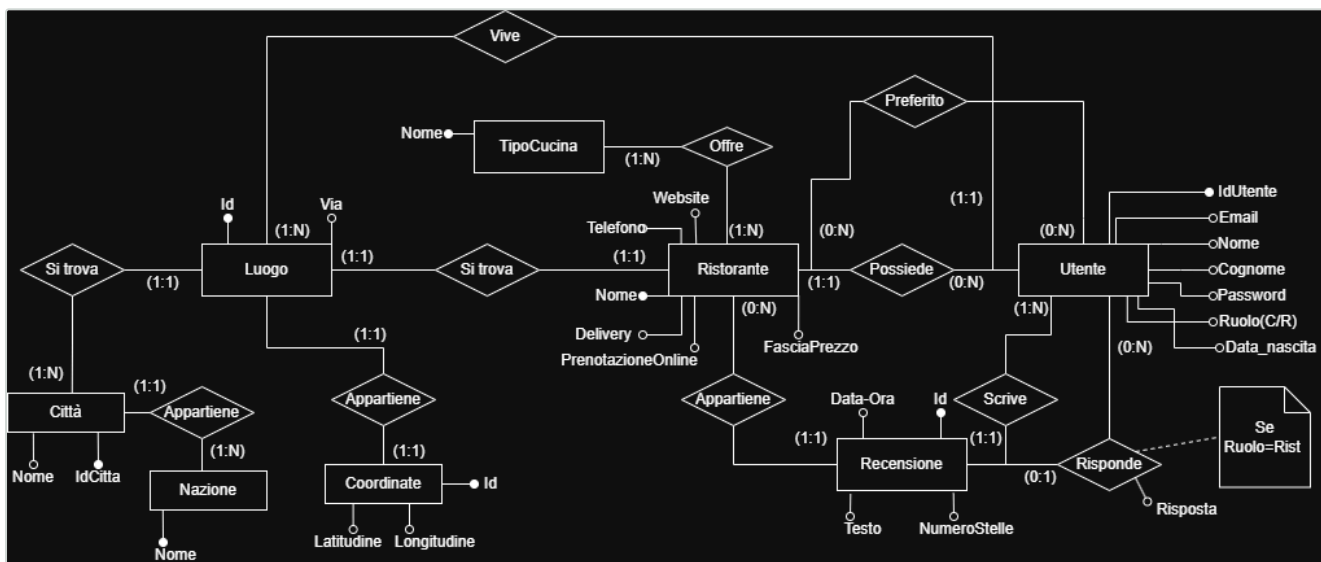
Relazione	Chiave e vincoli principali
NAZIONE, TIPO_CUCINA	Chiave primaria sul nome; tabelle di dominio popolate in importazione
COORDINATE	SERIAL ; latitudine DECIMAL(10,8) e longitudine DECIMAL(11,8)
CITTA, LUOGO	Chiavi surrogate; LUOGO.id_coordinate è UNIQUE , a realizzare la cardinalità uno a uno
UTENTE	email UNIQUE ; is_ristoratore discrimina il ruolo
RISTORANTE	id_luogo UNIQUE ; stelle_michelin vincolata da CHECK nell'intervallo 0-3
RECENSIONE	numero_stelle vincolata da CHECK nell'intervallo 1-5
RISPOSTA	Chiave primaria coincidente con id_recensione : al più una risposta per recensione
PREFERITO, RISTORANTE_TIPO_CUCINA	Chiave primaria composta dalle due chiavi esterne, che impedisce i duplicati

Vincoli in linguaggio naturale

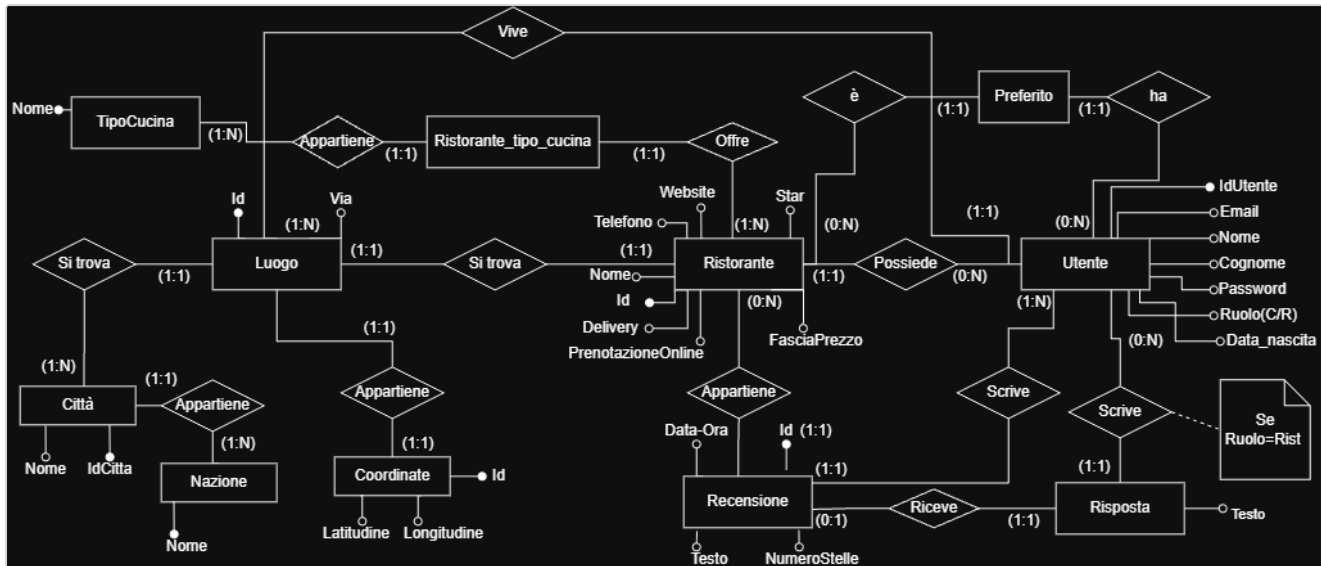
Alcune regole non sono esprimibili nello schema e restano affidate all'applicazione:

- il ruolo di un utente è esclusivo: o cliente o ristoratore, mai entrambi;
- solo un utente con ruolo di ristoratore può possedere ristoranti e rispondere alle recensioni;
- la risposta a una recensione la scrive il ristoratore proprietario del locale recensito, e una recensione già evasa non compare più fra quelle in attesa;
- un ospite può consultare il catalogo ma non lasciare recensioni né salvare preferiti;
- la città indicata in una ricerca deve esistere nel catalogo, altrimenti il filtro non restituisce nulla.

Gli altri vincoli sono già nello schema. Due CHECK limitano numero_stelle e stelle_michelin, l'email è unica e i legami uno a uno fra ristorante e luogo e fra luogo e coordinate sono garantiti da colonne UNIQUE. Le chiavi primarie composte, infine, impediscono di aggiungere due volte lo stesso preferito o la stessa cucina.



Schema ER non ristrutturato, derivato dai requisiti.



Schema ER ristrutturato, pronto per la traduzione logica.

Durante la progettazione del database per l'applicazione TheKnife, siamo partiti da uno schema concettuale per poi arrivare al modello logico definitivo, adottando alcune scelte pratiche per mantenere il sistema efficiente e facile da interrogare.

In primo luogo, abbiamo notato che «Clienti» e «Ristoratori» condividevano gli stessi identici dati anagrafici. Abbiamo deciso di accorparli in un'unica entità *Utente*, distinguendoli semplicemente tramite un attributo *Ruolo*. Per garantire la stessa pulizia, abbiamo isolato i dati geografici in entità distinte (*Città*, *Nazione*, *Coordinate*, *Luogo*): questa divisione ci ha permesso di evitare ridondanze nel database e di rendere i filtri di ricerca nell'applicazione molto più veloci e precisi.

Nel passaggio verso il modello logico (la fase di ristrutturazione per PostgreSQL), abbiamo dovuto riadattare lo schema ai limiti dei database relazionali. Tutte le relazioni «Multi-a-Multi», come l'assegnazione dei ristoranti preferiti o le varie tipologie di cucina offerte da un locale, sono trasformate in tabelle intermedie (*Preferito* e *Ristorante_tipo_cucina*).

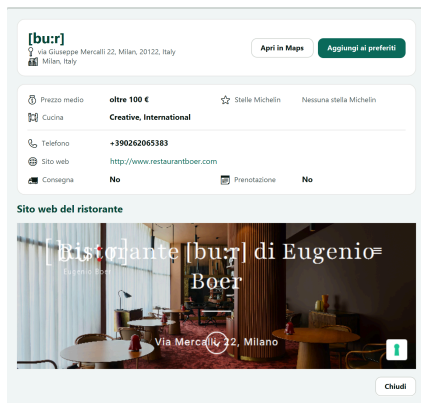
Un'ultima modifica importante ha riguardato le risposte alle recensioni. Nello schema iniziale la risposta figurava come un semplice attributo testuale; tuttavia, per mantenere le tabelle più pulite, evitare spazi vuoti nel database e preparare il sistema a espansioni future, abbiamo preferito creare un'entità indipendente (*Risposta*). Ora ogni risposta esiste come «oggetto» a sé stante, collegato in modo inequivocabile (1:1) alla sua recensione e al ristoratore che l'ha scritta.

2.2.7 Strutture dati utilizzate

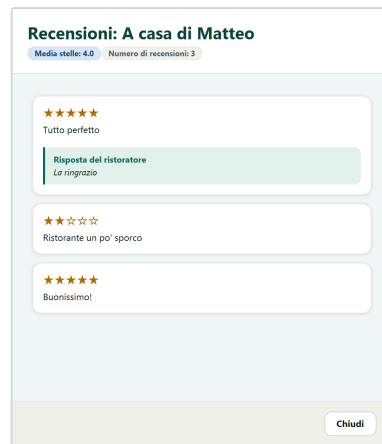
Con il passaggio al database le strutture in memoria non sono più l'archivio, ma il formato di trasporto e di presentazione. I DTO di `commonTK` implementano `Serializable` e dichiarano un `serialVersionUID` esplicito, così il contratto resta stabile fra i due lati; non contengono logica di dominio, a parte le validazioni che devono valere su entrambi, come `UtenteDTO.emailValida(String)`.

Le risposte multiple viaggiano come `LinkedList`, consumate in sequenza. Le richieste con due identificativi, per esempio l'aggiunta di un preferito, usano una `HashMap<String,Integer>` invece di un DTO dedicato. Sul client la schermata principale ha per sorgente una `ObservableList<RistoranteDTO>`: un `ChangeListener` ricostruisce la paginazione a ogni variazione, così chi manipola i dati non tocca i nodi grafici.

Città e cucine servono all'autocompletamento di quasi ogni maschera: `Utility` le chiede al server una volta sola e le conserva in due liste statiche, con accessi `synchronized` perché il caricamento avviene su thread diversi. Le statistiche delle schede non si calcolano in memoria: media e conteggio delle recensioni arrivano da sottointerrogazioni aggregate e le cucine da `ARRAY_AGG`, quindi una sola interrogazione popola il DTO per intero.



Scheda composta da un solo `RistoranteDTO`.



`LinkedList<RecensioneDTO>` resa in tabella.



Preferiti ottenuti con una join sulla tabella ponte.

2.2.8 Scelte algoritmiche

Costruzione dinamica dell'interrogazione di ricerca

Il filtro dei ristoranti è l'operazione più sollecitata. La query si costruisce a runtime concatenando le sole condizioni corrispondenti ai criteri valorizzati, a partire da un `WHERE 1=1` che rende uniforme l'aggiunta dei predicati. I valori non finiscono mai nella stringa: vengono accumulati in una lista e associati per posizione al `PreparedStatement`. Così l'*SQL injection* è esclusa e il DBMS può riusare il piano di esecuzione.

```
if (filtro.getFasciaPrezzo() != null && !filtro.getFasciaPrezzo().trim().isEmpty()) {
    sql.append("AND R.fascia_prezzo = ? ");
    parametri.add(filtro.getFasciaPrezzo().trim());
}
/* ... */
try (PreparedStatement ps = conn.prepareStatement(sql.toString())) {
    for (int i = 0; i < parametri.size(); i++) {
        ps.setObject(i + 1, parametri.get(i));
    }
}
```

Algoritmo di Haversine per il calcolo delle distanze

L'algoritmo di Haversine è utilizzato per calcolare la distanza tra due punti sulla superficie terrestre, assumendo la Terra come una sfera. Il calcolo si basa sulle coordinate geografiche, latitudine e longitudine, espresse in gradi.

Nell'applicazione serve alla ricerca per vicinanza, che non si ferma al confine comunale. Il DAO individua un ristorante di riferimento nella città richiesta, ne legge le coordinate e le conserva in un `record RiferimentoRicerca(String citta, double latitudine, double longitudine)`. Ogni riga del risultato viene accettata se appartiene alla stessa città oppure se dista meno di dieci chilometri dal riferimento.

```
double a = pow(sin(dLat / 2), 2) +
    cos(lat1Rad) * cos(lat2Rad) * pow(sin(dLon / 2), 2);
double c = 2 * atan2(sqrt(a), sqrt(1 - a));
double distanzaKm = RAGGIOTERRESTRE_KM * c;
```

Cifratura delle credenziali

La password non transita mai in chiaro sul socket e non viene memorizzata come tale: il client ne calcola il digest SHA-256 con `MessageDigest`, lo codifica in esadecimale e lo mette nell'`AuthDTO`. Il confronto in fase di accesso avviene quindi fra digest.

Ricerche asincrone e scarto delle risposte obsolete

Ogni interazione con il server è bloccante e gira in un `Task` JavaFX su un thread demone dedicato; i nodi grafici si aggiornano nei gestori `setOnSucceeded` e `setOnFailed`, notificati sul thread applicativo. A ogni ricerca è associato un contatore monotono: al completamento il risultato viene applicato solo se quel contatore è ancora l'ultimo emesso, così una risposta lenta relativa a un filtro superato non sovrascrive quella corrente.

```

long identificativoRicerca = ++ultimaRicerca;

Task<List<RistoranteDTO>> richiesta = new Task<>() {
    @Override
    protected List<RistoranteDTO> call() {
        return filtraRistoranti(filtro);
    }
};

richiesta.setOnSucceeded(evento -> {
    if (identificativoRicerca == ultimaRicerca) {
        mostraRistoranti(richiesta.getValue());
    }
});

```

Impaginazione a costruzione differita

L'elenco può contenere migliaia di elementi. Il controllo `Pagination` usa una *page factory* che costruisce le schede della sola pagina richiesta, dodici per volta: il costo di creazione dei nodi diventa indipendente dalla cardinalità del risultato.

2.2.9 Design pattern adottati

Pattern	Realizzazione	Motivazione
Singleton	<code>Session</code> , <code>GestioneRichieste</code>	Stato di autenticazione e connessione di rete unici per processo; l'accesso al secondo è <code>synchronized</code> e ricrea il socket se risulta chiuso
Data Access Object	Package <code>it.uninsubria.dao</code>	Isola il codice JDBC dal protocollo: <code>SlaveThread</code> non contiene istruzioni SQL
Data Transfer Object	Package <code>it.uninsubria.dto</code>	Riduce gli scambi trasportando grafi completi e definisce il contratto fra i moduli
MVC	FXML, controller, DTO	Separa descrizione della vista, logica di interazione e dati
Thread per connessione	<code>AppServer</code> e <code>SlaveThread</code>	Concorrenza fra client con codice sequenziale e leggibile nel servente
Observer	<code>ObservableList</code> , <code>ListChangeListener</code> , proprietà JavaFX	Disaccoppia la manipolazione dei dati dalla ricostruzione dell'interfaccia

`Finestre` merita una precisazione: oltre a caricare l'FXML e a costruire lo `Stage` modale, registra la scena presso `Temi`. Quest'ultimo tiene un elenco di `WeakReference<Scene>` e, quando il tema cambia, aggiunge o toglie il foglio di stile scuro su tutte le scene vive. Una finestra costruita a mano non risulta registrata e resta in tema chiaro.

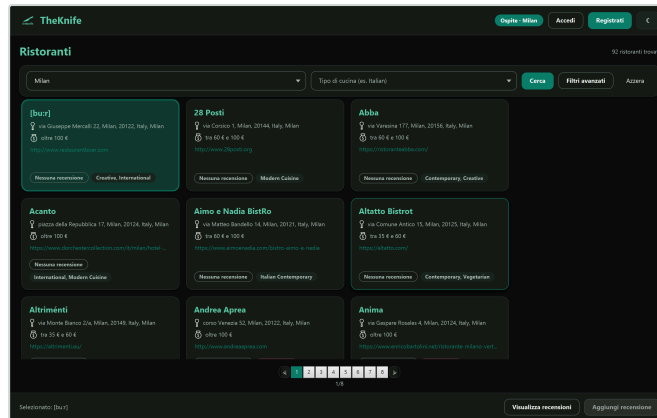
```

FXMLLoader caricatore = new FXMLLoader(urlFXML);
Parent radice = caricatore.load();
Scene scena = new Scene(radice);

Temi.registra(scena);

```

L'aspetto dell'applicazione sta in un unico foglio di stile, che dichiara la palette come *looked-up colors*; il tema scuro ne ridefinisce solo i valori. Negli FXML e nel codice Java non compaiono colori né dimensioni.

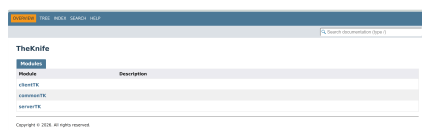


Tema scuro applicato per sola sostituzione dei token della palette.

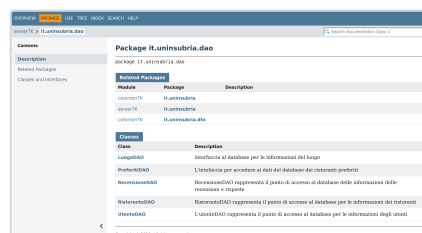
2.2.10 Documentazione Javadoc

La documentazione delle interfacce di programmazione è parte integrante del manuale tecnico. Ogni classe e ogni metodo pubblico portano un commento Javadoc con i tag `@param`, `@return` e `@throws` dove pertinenti, e i tag `@author` dei singoli autori, mantenuti a livello di metodo per tracciare la paternità del codice. La generazione è automatizzata dal `maven-javadoc-plugin` dichiarato nel POM padre: `./mvnw javadoc:aggregate` produce un unico sito statico in `doc/javadoc`, con i tre moduli come voci di primo livello.

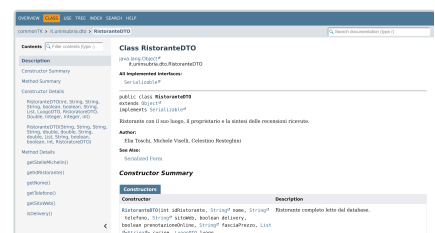
Prerequisito dell'aggregazione. L'obiettivo `javadoc:aggregate` rifiuta i progetti in cui convivono moduli JPMS con descrittore e moduli senza. Per questo tutti e tre i moduli dichiarano il proprio `module-info.java`, con il nome allineato a quello della cartella: anche la Javadoc generata da IntelliJ usa `--module-source-path` e pretende questa corrispondenza.



Sito aggregato: i tre moduli documentati.



Modulo `serverTK`: il package dei DAO.



Modulo `commonTK`: la classe `RistoranteDTO`.

2.3 Data set di test

Il collaudo si è basato sul catalogo della guida Michelin, circa 17.700 esercizi, integrato da utenti, recensioni e risposte creati apposta. I tre file CSV stanno fra le risorse del server, accanto alla migrazione, e vengono letti con OpenCSV, che gestisce i campi quotati contenenti separatori. L'importazione avviene una sola volta: se la tabella `RISTORANTE` è già popolata, la procedura viene saltata.

File	Tabelle popolate
<code>michelin_my_maps.csv</code>	NAZIONE, CITTA, COORDINATE, LUOGO, RISTORANTE, TIPO_CUCINA, RISTORANTE_TIPO_CUCINA
<code>users.csv</code>	UTENTE, PREFERITO
<code>recensioni.csv</code>	RECENSIONE, RISPOSTA

Le tabelle di dominio si popolano con `ON CONFLICT ... DO NOTHING`, che rende l'inserimento idempotente. Durante l'importazione la fascia di prezzo, che nel catalogo è una sequenza di simboli di euro, viene tradotta in un intervallo testuale da `Utility.ConvertiStringaPrezzo`, secondo la scala della guida Michelin.

Simbolo	Fascia di prezzo
€	meno di 35 €
€€	tra 35 € e 60 €
€€€	tra 60 € e 100 €
€€€€	oltre 100 €

Le prove hanno riguardato:

- la normalizzazione dei dati del CSV nelle undici relazioni, con attenzione alla deduplicazione di nazioni, città e tipi di cucina;
- il comportamento dei filtri combinati e la coerenza fra media, numero di recensioni e contenuto della scheda;
- i casi limite privi di dati — ristorante senza recensioni, utente senza preferiti, ristoratore senza locali, recensione senza risposta — e i campi facoltativi assenti nel catalogo, resi come colonne `NULL`.

Gli utenti del data set servono anche come credenziali di prova: `users.csv` contiene tre clienti e due ristoratori, elencati nel manuale utente.

3. Limiti della soluzione sviluppata

Assenza di un pool di connessioni

Ogni operazione apre una nuova connessione JDBC tramite `DriverManager` e la chiude al termine del blocco `try-with-resources`. La soluzione è semplice e non perde risorse, ma il costo di apertura pesa su ogni richiesta. Con molti client simultanei il limite di `max_connections` di PostgreSQL diventerebbe il primo collo di bottiglia: un pool di connessioni sarebbe l'evoluzione naturale.

Protocollo non tipizzato e privo di cifratura

Il protocollo si basa sulla serializzazione Java e su comandi scritti come stringhe letterali: la coerenza fra le due estremità non la verifica il compilatore, e un errore di battitura si manifesta solo a runtime. La comunicazione viaggia in chiaro, senza TLS e senza autenticazione del canale, quindi non è adatta a una rete non fidata. L'indirizzo del server, infine, è cablato su `localhost` e non si può cambiare senza ricompilare.

Gestione degli errori orientata alla diagnosi

Le eccezioni SQL finiscono sullo standard error del server e diventano una risposta vuota o nulla per il client, che non riceve un codice di errore strutturato: un guasto tecnico e un risultato legittimamente vuoto gli appaiono uguali.

Cifratura delle credenziali

Le password sono conservate come digest SHA-256. La scelta è adeguata al contesto didattico, ma in un sistema reale converrebbe una cifratura più robusta, con algoritmi pensati per le password come bcrypt o Argon2.

Assenza di notifiche e di modalità non in linea

Il protocollo è interamente a domanda e risposta: il server non può avvisare i client, e le modifiche di un utente diventano visibili agli altri solo ripetendo la richiesta. Il client non ha alcuna cache locale e senza server non mostra nulla.

Copertura di test automatici assente

Il progetto non contiene test dell'applicazione: la verifica è stata condotta a mano sul data set descritto nel paragrafo 2.3.

4. Sitografia / Bibliografia

- Oracle, *Java SE 21 API Specification*, Online: <https://docs.oracle.com/en/java/javase/21/docs/api/index.html>
- Oracle, *Java Object Serialization Specification*, Online: <https://docs.oracle.com/en/java/javase/21/docs/specs/serialization/>
- OpenJFX, *JavaFX Documentation*, Online: <https://openjfx.io>
- OpenJFX, *JavaFX CSS Reference Guide*, Online: <https://openjfx.io/javadoc/21/javafx.graphics/javafx/scene/doc-files/cssref.html>
- Apache Software Foundation, *Maven – Guide to Working with Multiple Modules*, Online: <https://maven.apache.org/guides/mini/guide-multiple-modules.html>
- PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*, Online: <https://www.postgresql.org/docs/>
- PostgreSQL JDBC, *pgJDBC Documentation*, Online: <https://jdbc.postgresql.org>
- Redgate, *Flyway Documentation*, Online: <https://documentation.red-gate.com/flyway>
- OpenCSV, *OpenCSV Users Guide*, Online: <https://opencsv.sourceforge.net>
- Object Management Group, *Unified Modeling Language Specification 2.5.1*, Online: <https://www.omg.org/spec/UML/>
- diagrams.net, *Diagram Editor*, Online: <https://app.diagrams.net>
- JetBrains, *Javadoc in IntelliJ IDEA*, Online: <https://www.jetbrains.com/help/idea/javadoc.html>
- Michelin, *Guida Michelin*, Online: <https://guide.michelin.com>